SummitUp:
Technical Architecture &
Final Project Report

Course: Mobile Programming
Developer: Muhammad Haziq Bin Muhamad Nur Azmin
Student ID: 202414525

1. Project Overview & Motivation

1.1 Project Definition

SummitUp is an offline-first native Android mobile application designed to enable hikers to document, review, and share detailed hiking journals, trail routes, and photography within a trusted community and among peers.

1.2 Problem Statement & Motivation

- Repetitive Inquiries & Lost Details: Friends and outdoor peers constantly ask for specific trail recommendations, preparation requirements, and difficulty levels. Over time, hikers frequently forget nuanced route details, waypoints, and trail conditions.
- Lack of Social Trust in Generic Apps: Mainstream commercial applications (such as AllTrails) offer crowd-aggregated data but lack the personal storytelling, context, and social credibility that comes from direct recommendations from friends.
- Connectivity Dead Zones: Mountain trails frequently lack reliable cellular coverage. Traditional online-only hiking applications become unusable or lose uncommitted journal logs when hikers lose connectivity.

1.3 Target Audience

- Outdoor enthusiasts, amateur hikers, and trail runners.
- Hiking clubs and social outdoor communities seeking to share verified route experiences.
- Beginner hikers looking for firsthand, trustworthy trail accounts, equipment checklists, and elevation insights from friends.

1.4 Core Value Proposition

SummitUp eliminates the need to repeatedly explain trail logistics to friends. By recording route coordinates, rich-text trail notes, and photos into a structured journal that functions seamlessly offline and syncs automatically online, users can create and distribute interactive hiking guides with a single click.

2. System Architecture & Technology Stack

2.1 Core Environment & Language

- Programming Language: 100% Kotlin (utilizing Kotlin DSL .kts for all build configurations).
- Target Platforms:
    - Minimum SDK: Android 15 (API 35)
    - Target SDK: Android 16 (API 36).
- Build System: Gradle with Version Catalogs (libs.versions.toml) for centralized, type-safe dependency management.
- Concurrency Engine: Kotlin Coroutines and Kotlin Flow for asynchronous local database transactions, network calls, and reactive UI streaming.

2.2 Core Frameworks & Integrations

- UI & Presentation:
    - Jetpack Compose: Modern declarative UI framework.
    - Material Design 3 (M3): Standardized UI components including CenterAlignedTopAppBar, FilterChip, and Scaffold.
    - RichEditor Compose: Markdown/rich-text rendering allowing users to style hike descriptions with bold, italic, and formatted sizing.
- Type-Safe Navigation:
    - Implements Compose Navigation with Type-Safe Routes (@Serializable Kotlin data classes and objects) instead of legacy string-based route paths.
- Persistence & Data Layer:
    - Local Storage (Room Database): SQLite abstraction managing local offline caching, complex queries, and entity relationships (e.g., HikeWithPhotos).
    - Remote Storage (Firebase Firestore): Cloud NoSQL database hosting the global social discovery feed, public journals, and user profiles.
- Media Management & CDN:
    - Cloudinary Android SDK: External image CDN for compressing, storing, and delivering high-resolution hike photographs (bypassing Firebase Storage overhead).
    - Coil 3: Asynchronous image loading and disk/memory caching for user avatars and hike photo galleries.
- Mapping, Geocoding & Location:
    - Naver Map Compose SDK: Native map rendering and trail pin visualization tailored for Korean geographic detail.
    - Android Geocoder API & Location Services: Utilizes ACCESS_FINE_LOCATION and ACCESS_COARSE_LOCATION with foreground location providers to convert search queries into precise coordinates and capture GPS coordinates.
- Authentication & Identity:
    - Android Credentials Manager (androidx.credentials): Modern Android identity framework supporting Passkeys and Google Sign-In.
    - Firebase Authentication: Handles backend session verification for both email/password credentials and Google Auth.

- Universal Deep Linking & Web Fallback:
    - Verified Android App Links (android:autoVerify="true"): Configured for the domain summitup-c4b4a.web.app. Intercepts web URLs to route users directly into the specific hike detail screen in the native app.
    - Firebase Hosting: Hosts a lightweight single-page web preview displaying the hike overview, photos, and a map placeholder when shared with non-app users.

3. Engineering Architecture & Design Patterns

3.1 Model-View-ViewModel (MVVM) & Unidirectional Data Flow (UDF)

SummitUp strictly follows Google's recommended architecture principles:

- View (UI Layer): Pure, declarative Jetpack Compose composables that observe state and emit user events.
- ViewModel Layer: Encapsulates business logic, managing state persistence across lifecycle events. ViewModels expose a single immutable UiState via Kotlin StateFlow and handle interactions via lambda-based UiActions.
- Model Layer: Complete separation between remote Data Transfer Objects (HikeDto), local database tables (HikeEntity), and UI display models (HikeWithPhotos). Dedicated mapper functions transform data across boundaries.

3.2 Offline-First Synchronization Strategy

To ensure uninterrupted usage in remote outdoor environments:

1. When a user creates a hike entry, all metadata and local image URIs are committed immediately to the local Room Database with a sync flag (isSynced = false).
2. When an active network connection is detected, background Coroutines upload images to Cloudinary, retrieving secure remote URLs.
3. The complete record with remote media URLs is pushed to Firebase Firestore.
4. Upon successful cloud acknowledgment, the local Room database updates the record to isSynced = true.
5. If the network drops at any point, un-synced entries remain locally accessible and retry automatically once connectivity is re-established.

3.3 Single Source of Truth (Repository Pattern)

HikeRepository and AuthRepository act as the single source of truth for the application. ViewModels request data exclusively through repositories, which orchestrate whether to serve cached data from Room or fetch live updates from Firestore.

4. Key Feature Implementations & Technical Details

4.1 Social Discovery Feed with Reactive Coroutine Streams

The discovery feed displays public hikes shared across the community. Instead of polling the server, SummitUp utilizes Kotlin callbackFlow to adapt Firebase Firestore's addSnapshotListener into a cold asynchronous reactive stream:

- Real-time additions, deletions, or edits on the remote database trigger instant UI re-compositions in the Compose LazyColumn.
- Author profile data is cached locally to prevent redundant network fetches for repeated authors across the feed.



```
private fun loadHomeFeed() {
    viewModelScope.launch {
        isLoading = true

        // 1. Collect from Firestore Flow
        hikeRepository.getPublicHikesFlow().collectLatest { remoteHikes ->

            // 2. Map DTOs to UI State and ensure authors are cached locally
            val combinedList = remoteHikes.map { dto ->
                // Check local cache first
                val localUser = authRepository.getLocalUserSync(dto.userId)

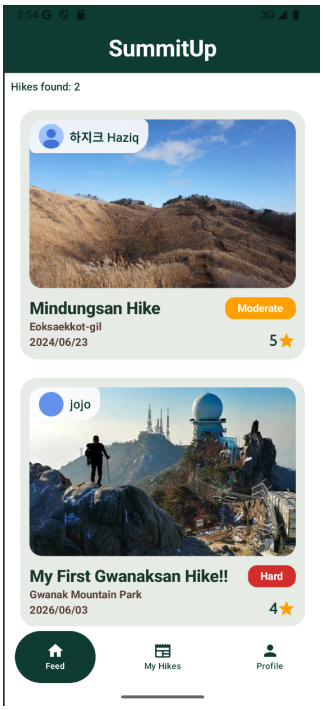
                // ONLY fetch from network if user is not in Room
                if (localUser == null  && !fetchingUsers.contains(dto.userId)) {
                    fetchingUsers.add(dto.userId)
                    launch {
                        try {
                            // Download from Firestore to Room
                            val result = authRepository.getRemoteUser( targetId = dto.userId)
                            // If download was successful, update the UI state
                            if (result.isSuccess) {
                                val freshlyDownloadedUser =
                                    authRepository.getLocalUserSync(dto.userId)
                                if (freshlyDownloadedUser != null) {
                                    // Trigger recomposition for this specific user
                                    updateAuthorInList(dto.userId,  userEntity = freshlyDownloadedUser)
                                }
                            }
                        } finally {
                            fetchingUsers.remove( element = dto.userId)
                        }
                    }
                }

                // Convert DTO to UI model
                HikeWithPhotosWithUser(
                    hikeWithPhotos = dto.toHikeWithPhotos(),
                    user = localUser
                )
            }
            hikes = combinedList
            isLoading = false
        }
    }
}
```

```kotlin
// Direct stream from Firestore.
// Use callbackFlow to get real-time updates for the public feed.
1 Usage
fun getPublicHikesFlow(): Flow<List<HikeDto>> = callbackFlow {
    // Reference to your firestore collection
    val query = firestore.collection( collectionPath = "hikes")
        .whereEqualTo( field = "isPrivate", value = false)
        .orderBy( field = "timestamp", direction = Query.Direction.DESCENDING)
        .limit(10) // Limit to 10 for performance

    val subscription = query.addSnapshotListener { snapshot, error ->
        if (error != null) {
            println("Firestore Error: ${error.message}")
            return@addSnapshotListener
        }

        if (snapshot != null) {
            val hikes = snapshot.toObjects( clazz = HikeDto::class.java)
            trySend( element = hikes)
        }
    }
    awaitClose { subscription.remove() }
}
```



4.2 Cross-Platform Hike Sharing & Verified App Links

When a user shares a hike entry via WhatsApp, KakaoTalk, or SMS:

- The app dispatches an Android Intent.ACTION_SEND containing a universal link:
- https://summitup-c4b4a.web.app/hike/{hikeId}
- If the recipient has SummitUp installed, Android App Links intercepts the URL and navigates directly to HikeDetail(hikeId) inside the app.
- If the recipient does not have the app, the URL loads a responsive web page on Firebase Hosting that presents a preview card with a prompt to launch or install the native application.

```
<intent-filter android:autoVerify="true">
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />

    <!-- Accepts https://summitup-app.web.app/hike/{id} -->
    <data android:scheme="https" />
    <data android:host="summitup-c4b4a.web.app" />
    <data android:pathPrefix="/hike" />
</intent-filter>
```

```
fun onShare(context: Context) {

    val hike = hikeWithPhotosWithUser?.hikeWithPhotos?.hike ?: return

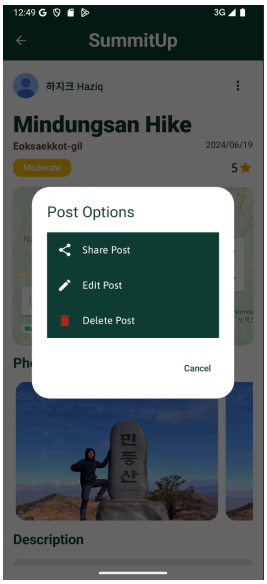
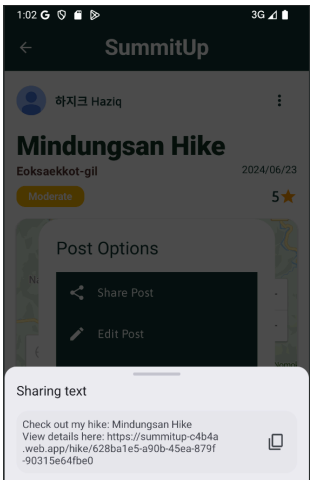
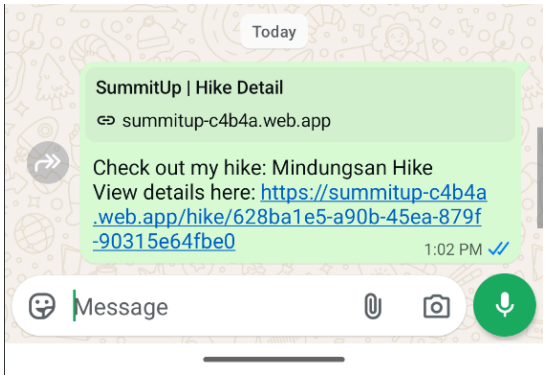
    // Replace with your actual Firebase Hosting URL

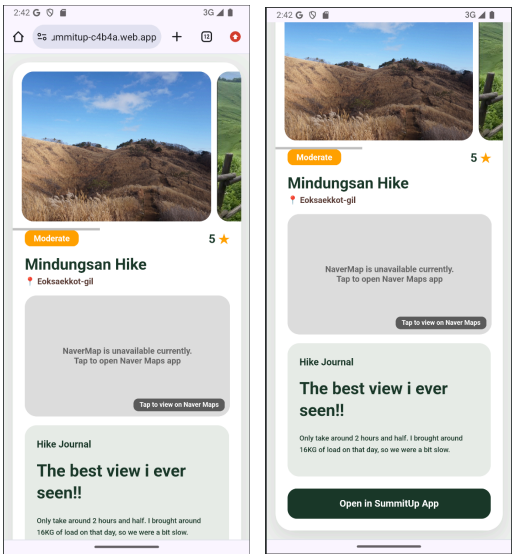
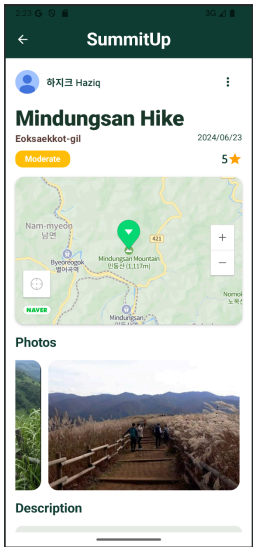
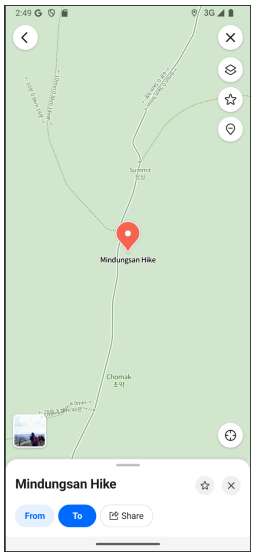
    val shareUrl = "https://summitup-c4b4a.web.app/hike/${hike.hikeId}"


    val sendIntent: Intent = Intent().apply {
        action = Intent.ACTION_SEND
        putExtra( name = Intent.EXTRA_TEXT, value = "Check out my hike: ${hike.title}\nView details here: $shareUrl")
        type = "text/plain"
    }


    val shareIntent = Intent.createChooser( target = sendIntent, title = "Share Hike")
    context.startActivity( p0 = shareIntent)
}
```

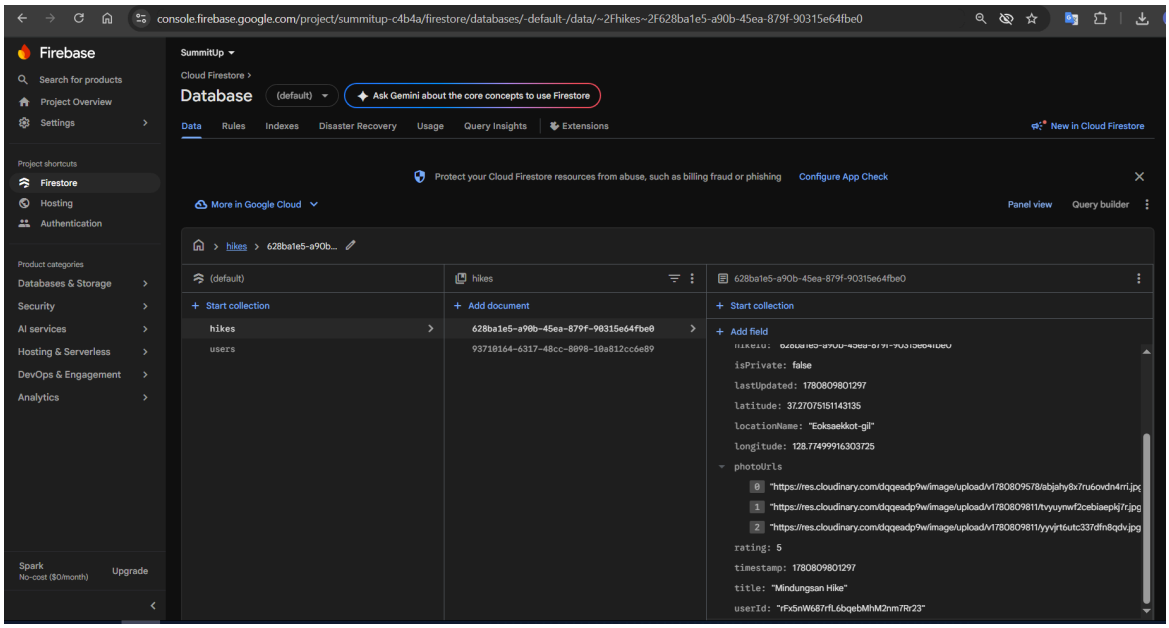
```
<a href="intent://summitup-c4b4a.web.app/hike/${id}#Intent;scheme=https;package=com.hziqzmin.summitup;end" class="btn-app">
    Open in SummitUp App
</a>
```

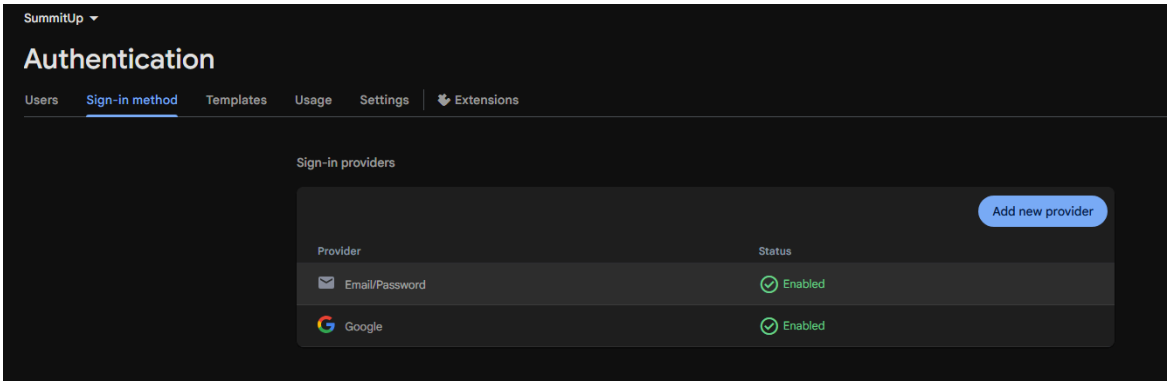
	Users click the share button.
	Link to the web-app hosted on Firebase Hosting. Go to the web-page using any browser.
	Or copy the link and share it with your friends.

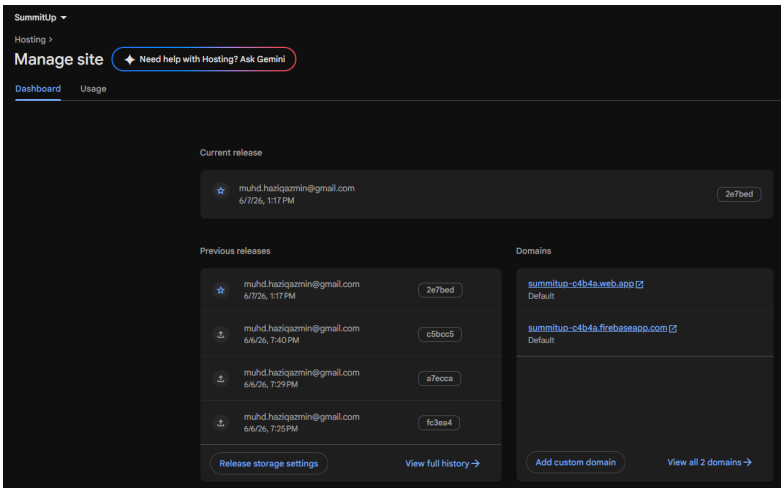
	View the journal post on the web-page for people who are not using SummitUp mobile app.
	If they do have the app, clicking the previous link will directly bring them to the page in the SummitUp app.
	By clicking the map, it will direct them straight to Naver Map mobile app and show the location in that app so users can navigate to that location.

4.3 Cloud Data Synchronization, User Authentication and Hosting

Utilized Firebase to store data online (Firestore DB), handle user authentication (Auth) and host a preview webpage on Hosting.








```kotlin
suspend fun signUp(email: String, password: String, username: String): Result<Unit> {
    return runCatching {
        // 1. Check if username is already taken in Firestore
        val usernameQuery = firestore.collection( collectionPath = "users")
            .whereEqualTo( field = "username", value = username)
            .get()
            .await()

        if (!usernameQuery.isEmpty) {
            throw Exception("USERNAME_TAKEN")
        }

        // 2. Proceed with Firebase Auth
        val authResult = firebaseAuth.createUserWithEmailAndPassword( p0 = email, p1 = password).await()
        val userId = authResult.user?.uid ?: throw Exception("AUTH_FAILED")

        // 3. Prepare the User Model
        val userEntity = UserEntity(
            userId = userId,
            username = username,
            email = email,
            isSynced = true
        )

        // 4. Save to Remote Firestore
        val userDto = UserDto.fromEntity(userEntity)
        firestore.collection( collectionPath = "users").document( documentPath = userId).set(userDto).await()

        // 5. Cache locally in Room (Offline-First requirement)
        userDao.upsertUser(userEntity)
    }
}
1 Usage
suspend fun signIn(email: String, password: String): Result<Unit> {
    return runCatching {
        val authResult = firebaseAuth.signInWithEmailAndPassword( p0 = email, p1 = password).await()
        val userId = authResult.user?.uid ?: throw Exception("LOGIN_FAILED")

        // Fetch latest profile from Firestore to populate Room cache
        fetchAndCacheRemoteUser(userId)

        // Initial sync on login
        hikeRepository.fetchMyHikesFromFirestore( authRepository = this)
        refreshUserStats(userId)
    }
}
```

5. Engineering Challenges & Troubleshooting

5.1 Two-Way Synchronization Complexity (Room DB & Firestore)

- Challenge: Handling offline record mutations without causing race conditions, duplicate entries, or data corruption when network connectivity fluctuates.
- Resolution: Implemented explicit synchronization state flags (isSynced) on the local entity schema. Designed the repository to write locally first, defer remote network synchronization to background Coroutines, and commit state updates within an atomic transaction.

5.2 Decoupled Media Upload Pipeline (Cloudinary Integration)

- Challenge: Uploading large image payloads directly to Firestore exceeds document limits and creates slow network bottlenecks.
- Resolution: Integrated the Cloudinary Android SDK into an asynchronous staging pipeline. Images are uploaded to Cloudinary asynchronously, transformed into optimized web delivery URLs, and only those lightweight URL strings are written into Firestore documents.

5.3 Credential & API Key Security

- Challenge: Preventing external API tokens (Naver Map Client ID, Cloudinary API keys, Firebase secrets) from leaking into version control.
- Resolution: Configured the secrets-gradle-plugin to inject sensitive keys into BuildConfig at compile-time from a local secrets.properties file that is strictly ignored by Git.

6. Project Retrospective & Future Roadmap

6.1 Key Competencies Developed

- Mastered modern Android declarative UI architecture using Jetpack Compose and Material 3.
- Implemented clean architectural boundaries using MVVM, Unidirectional Data Flow, and Repository patterns.
- Gained practical experience in hybrid storage architectures combining local caching (Room) with cloud backends (Firestore and Cloudinary).
- Built production-grade linking systems with Universal Deep Links and Firebase Hosting.

6.2 Future Enhancements

- Interactive Web Map Preview: Expand the single-page Firebase Hosting preview to render dynamic interactive map markers for users without the app.
- GPS Polyline Trail Recording: Enhance foreground location services to track and render full continuous route polylines and elevation charts rather than single coordinate pins.
- Strava API Integration: Integrate the Strava REST API to allow hikers to synchronize heart rate, pacing, and elevation gain statistics directly into their SummitUp hike journals.

7. Requirement Compliance Matrix

No	Project Requirement	Implementation Details	Status
1	Screen Transitions (Minimum 4)	Implemented 7 distinct screens via type-safe Compose Navigation	Exceeded
2	List Displays (Minimum 1)	2 LazyColumn feeds: Public Discovery Feed & My Hikes Journal	Exceeded
3	Image Handling	Cloudinary SDK remote image hosting + Coil 3 asynchronous caching	Fulfilled
4	Database Persistence	Dual-layer persistence: Local Room DB + Remote Firebase Firestore	Fulfilled
5	External APIs & Cloud Services	Firebase (Auth, Firestore, Hosting), Naver Map SDK, Cloudinary API	Fulfilled